TESTE DE SOFTWARE

Análise de Eficácia de Testes com Teste de Mutação

Avaliação e melhoria de uma suíte Jest com StrykerJS

Disciplina: Teste de Software

Trabalho: Análise de Eficácia de Testes com Teste de Mutação

Aluno: Rafael Ganascini de Moura

Matrícula: 858012

Repositório: https://github.com/RafaelMouraG/operacoes-mutante

1. Análise Inicial

O projeto é uma biblioteca com 50 funções matemáticas (`src/operacoes.js`) e uma suíte Jest com 50 testes, um por função. Todos passaram na execução inicial. A cobertura foi medida com `npm test -- --coverage` e a mutação com `npx stryker run` (StrykerJS 10 com `jest-runner`):

Statements	Branches	Functions	Lines	Mutation Score
85,41%	58,82%	100%	98,64%	**73,71%**

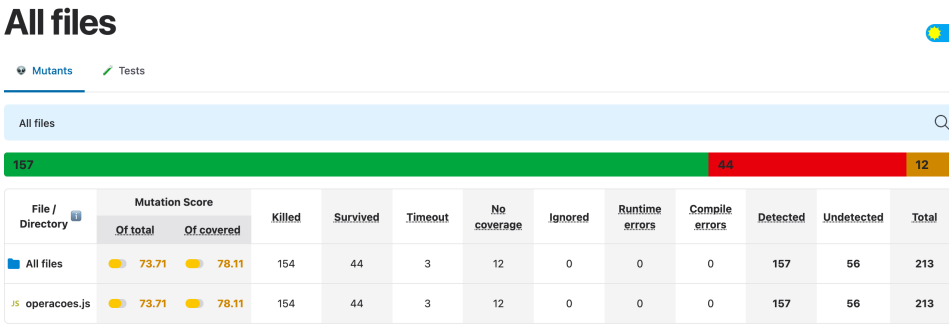


Figura 1 – Primeira execução do Stryker: 213 mutantes, 157 detectados, 44 sobreviventes e 12 sem cobertura

Discussão da discrepância. Pela cobertura, a suíte parece quase completa: 100% das funções e 98,64% das linhas são executadas. Ainda assim, 56 dos 213 mutantes (26%) não foram detectados, porque a cobertura mede o código **executado**, não o **verificado**. A suíte tem três problemas que a cobertura não mostra:

- **Só o "caminho feliz" é testado:** nenhum teste verifica casos falsos (`isPar(7)`), fronteiras (`isMaiorQue(5, 5)`) ou entradas vazias (`medianaArray([])`).
- **Entradas neutras escondem os bugs:** `celsiusParaFahrenheit(0)` anula qualquer alteração em `celsius * 9/5`.
- **Asserções fracas:** `toThrow()` sem mensagem só verifica que *algum* erro foi lançado, não qual.

A cobertura de branches (58,82%) já indicava o problema, mas só o teste de mutação mostra quais asserções faltam.

2. Análise de Mutantes Críticos

Mutante 1: remoção da ordenação em medianaArray (MethodExpression, linha 109)



Figura 2 – .sort((a, b) => a - b) removido em medianaArray

O que a mutação fez: removeu o `.sort(...)`, de modo que a mediana passa a ser calculada sobre o array na ordem original. **Por que sobreviveu:** o único teste era `medianaArray([1, 2, 3, 4, 5])`, um array **já ordenado**, em que ordenar ou não dá o mesmo resultado (`3`). Pelo mesmo motivo sobreviveram os comparadores trocados por `a + b` e `() => undefined`. Como o teste só usava um array de tamanho **ímpar**, o ramo de tamanho par (linhas 111–112) também nunca foi verificado.

Mutante 2: fronteira em isMaiorQue (EqualityOperator, linha 104)

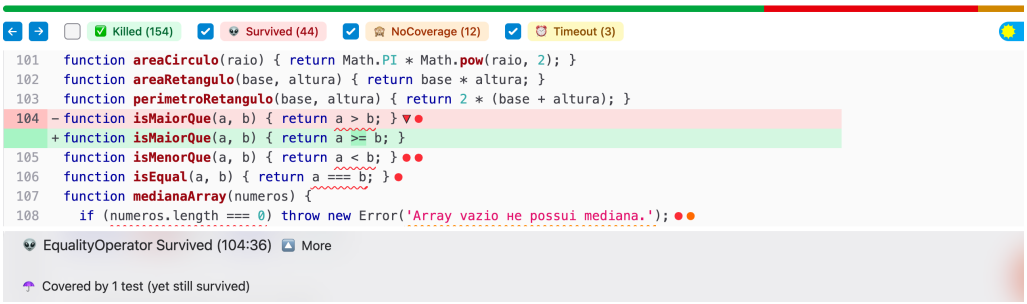


Figura 3 – a > b trocado por a >= b em isMaiorQue

O que a mutação fez: trocou `>` por `>=`, o clássico erro de *off-by-one*. **Por que sobreviveu:** o único teste era `isMaiorQue(10, 5) === true`, e para esses valores os dois operadores concordam. Só o caso `a === b` os diferencia, e ele não era testado. Também sobreviveu `return true`, porque nenhum teste esperava `false`. O mesmo padrão se repetiu em `isMenorQue`, `isEqual`, `isPar`, `isImpar` e `isDivisivel`.

Mutante 3: entrada neutra em celsiusParaFahrenheit (ArithmeticOperator, linha 93)

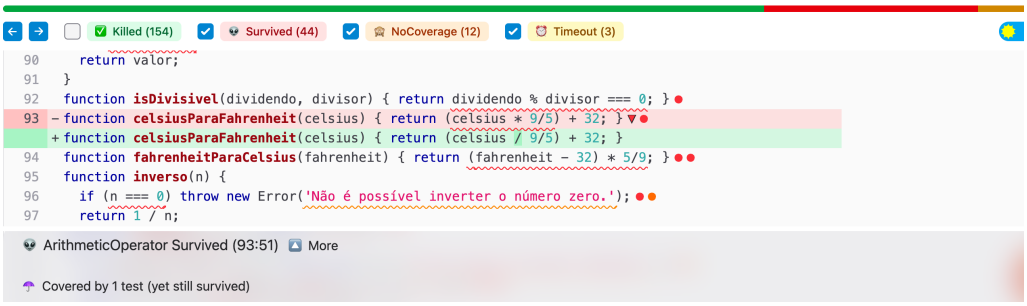


*Figura 4 – celsius * 9/5 trocado por celsius / 9/5 em celsiusParaFahrenheit*

O que a mutação fez: trocou a multiplicação por divisão (também sobreviveu `celsius * 9 * 5`). **Por que sobreviveu:** o teste usava `celsiusParaFahrenheit(0)`. Como **0 é o elemento absorvente da multiplicação**, todas as variantes valem `0` e o resultado continua `32`: o teste só verificava a constante `+ 32`. O mesmo ocorreu em `fahrenheitParaCelsius(32)`, que zera o termo `(fahrenheit - 32)`.

Outros padrões: mensagens de erro trocadas por `""` sobreviveram por causa do `toThrow()` genérico, e em seis funções o caso de erro nem era executado (NoCoverage). A remoção da guarda `if (numeros.length === 0) return 0` em `mediaArray` sobreviveu porque nunca se testou `mediaArray([])`. Em `isPrimo` e `clamp`, os ramos `return false`, `return min` e `return max` nunca foram verificados.

3. Solução Implementada

Os 50 testes originais foram mantidos e acrescentei um bloco `describe` com **21 novos testes** (71 no total). Para os mutantes analisados:

- **Mutante 1:** `medianaArray([5, 1, 3]) === 3`. Sem ordenação, o elemento central é `1`, e o comparador `a + b` mantém o array como está, o que também gera `1`. Os arrays pares `[1, 2, 3, 4]` e `[4, 1, 3, 2]` (mediana `2.5`) verificam a linha 112 e matam `mid - 1 → mid + 1` (que daria `3.5`), `+ → -` e `/ 2 → * 2`.
- **Mutante 2:** `isMaiorQue(5, 5) === false` é exatamente o ponto em que `>` e `>=` divergem, e `isMaiorQue(5, 10) === false` mata o `return true`.

- **Mutante 3:** `celsiusParaFahrenheit(100) === 212` exercita o fator `9/5` (com `/` daria ≈ 34,2 e com `* 9 * 5` daria 4532). O ponto `-40`, onde as escalas coincidem, confirma a fórmula com valor negativo.

Demais testes adicionados, cada um ligado a uma fraqueza encontrada:

Fraqueza	Testes adicionados
Só o caso `true` era testado	`isPar(7)`, `isImpar(4)`, `isDivisivel(10, 3)`, `isMenorQue(5, 5)`, `isEqual(7, 8)` → `false`
Fronteiras e ramos não executados	`raizQuadrada(0)`, `fatorial(0)`, `fatorial(1)`, `isPrimo(0..2)`, `isPrimo(4)`, `isPrimo(9)`, `clamp` abaixo, acima e nos limites
Entradas vazias	`mediaArray([])`, `produtoArray([])`; erro em `maximoArray([])`, `minimoArray([])`, `medianaArray([])`
Asserção de erro genérica	`toThrow('<mensagem exata>')` nas 7 funções que lançam erro
Entrada neutra	`fahrenheitParaCelsius(212) === 100` e `fahrenheitParaCelsius(-40) === -40`

Mutantes equivalentes

Depois dos novos testes restaram **7 sobreviventes** (96,71%). Todos são **mutantes equivalentes**: o código mutado se comporta igual ao original para *qualquer* entrada, então nenhum teste pode matá-los.

Linha	Mutação	Por que é equivalente
19 (fatorial)	`if (n === 0 || n === 1)` → `false`, `&&` e variantes (4)	Sem o `if`, o laço não executa para n = 0 ou 1 e a função já retorna 1
84 (produtoArray)	`if (numeros.length === 0)` → `false`	`[].reduce(fn, 1)` já retorna 1
88 e 89 (clamp)	`valor < min` → `<=` e `valor > max` → `>=`	No limite, retornar o limite ou o próprio valor dá o mesmo resultado

Eles foram marcados no código com o comentário `// Stryker disable next-line <Mutador>: <justificativa>`, sem alterar a lógica, e passam a contar como *Ignored* (11 no total, porque a diretiva também cobre 4 mutantes já mortos nessas linhas). Os testes de fronteira continuam na suíte garantindo o comportamento. Os `if` das linhas 19 e 84 são, na verdade, **código redundante**.

4. Resultados Finais

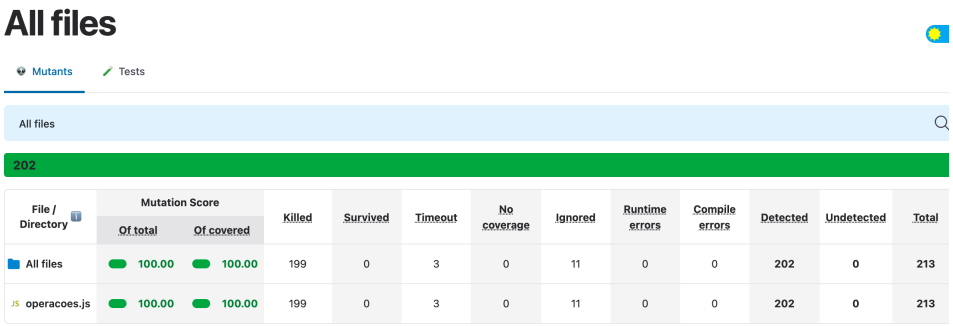


Figura 5 – Execução final do Stryker: 202 detectados, 0 sobreviventes, 11 ignorados

Métrica	Inicial	Final
Nº de testes	50	71
Cobertura de statements / branches / linhas	85,41% / 58,82% / 98,64%	100% / 100% / 100%
Mortos + timeout	157	202
Sobreviventes / sem cobertura	44 / 12	0 / 0
Ignorados (equivalentes)	0	11
Mutation Score	**73,71%**	**100,00%**

Sem excluir os equivalentes, a pontuação seria 96,71% (206 de 213), e os 7 restantes não podem ser mortos por nenhum teste. Considerando só os mutantes válidos, a suíte detecta **todas** as mutações, superando a meta de 98%. Os relatórios completos estão em `docs/inicial/` e `docs/final/`.

5. Conclusão

O trabalho mostrou na prática que **cobertura de código não mede a qualidade dos testes**: uma suíte com 98,64% de cobertura de linhas deixava passar 1 em cada 4 bugs simulados. A cobertura responde "esse código foi executado?", enquanto o teste de mutação responde "se esse código estivesse errado, algum teste falharia?", e só a segunda pergunta mede a eficácia da suíte.

O Stryker também orientou a melhoria: cada sobrevivente apontou uma fraqueza concreta (fronteira não testada, caso `false` esquecido, entrada neutra ou asserção genérica), o que permitiu escrever testes direcionados. Os mutantes equivalentes mostraram que a ferramenta exige julgamento crítico, já que nem todo sobrevivente indica um teste faltando. O custo é maior que o da cobertura (213 variações executadas contra a suíte), mas é a métrica mais confiável para saber se os testes protegem o sistema contra regressões.